

Kubernetes

Formation 2 jours — Jour 1

Architecture, Installation et Utilisation

Du conteneur à l'orchestration en production

Programme des 2 jours

Jour	Thèmes
Jour 1	Introduction, Architecture (composants, CRI/CNI/CSI, certificats), Installation (KaaS, manuelle, outils, HA), Utilisation (API, ressources, Services, ressources avancées, Network Policies, requests/limits)
Jour 2	Administration (déploiement, ressources, troubleshooting, autoscaling, bonnes pratiques), Enrichissement (Helm, Metrics Server, Prometheus/Grafana, CertManager, EFK), Service Mesh Istio, Écosystème Kubernetes

“ **Environnement** : cluster Kind multi-nœuds (1 control-plane + 3 workers) ”

Introduction

Pourquoi Kubernetes ?

Avant : VMs lourdes, scripts ad hoc, déploiements manuels, scaling pénible.

Conteneurs : portabilité, immuabilité, démarrage rapide... mais comment les gérer à l'échelle ?

```
1 conteneur          100 conteneurs          10 000 conteneurs  
docker run           docker-compose           ??? il faut un orchestrateur  
→                   →
```

Pourquoi Kubernetes ?

Kubernetes répond à :

- Déployer des centaines de conteneurs sur des dizaines de nœuds
- Auto-réparation en cas de crash (redémarrer, replanifier)
- Mise à l'échelle horizontale (HPA) et verticale (VPA)
- Mises à jour sans interruption (Rolling, Blue/Green, Canary)
- Découverte de services et load balancing intégrés
- Stockage persistant abstrait (CSI)
- Sécurité (RBAC, NetworkPolicies, SecurityContext)

Histoire et positionnement

Année	Événement
2003-2014	Google développe Borg puis Omega (internes)
2014	Open-sourcing de Kubernetes par Google
2015	Don à la CNCF (Cloud Native Computing Foundation)
2018	Premier projet gradu� de la CNCF
Aujourd'hui	Standard de facto de l'orchestration de conteneurs

Concurrents historiques (largement d pass s) : Docker Swarm, Mesos/Marathon, Nomad.

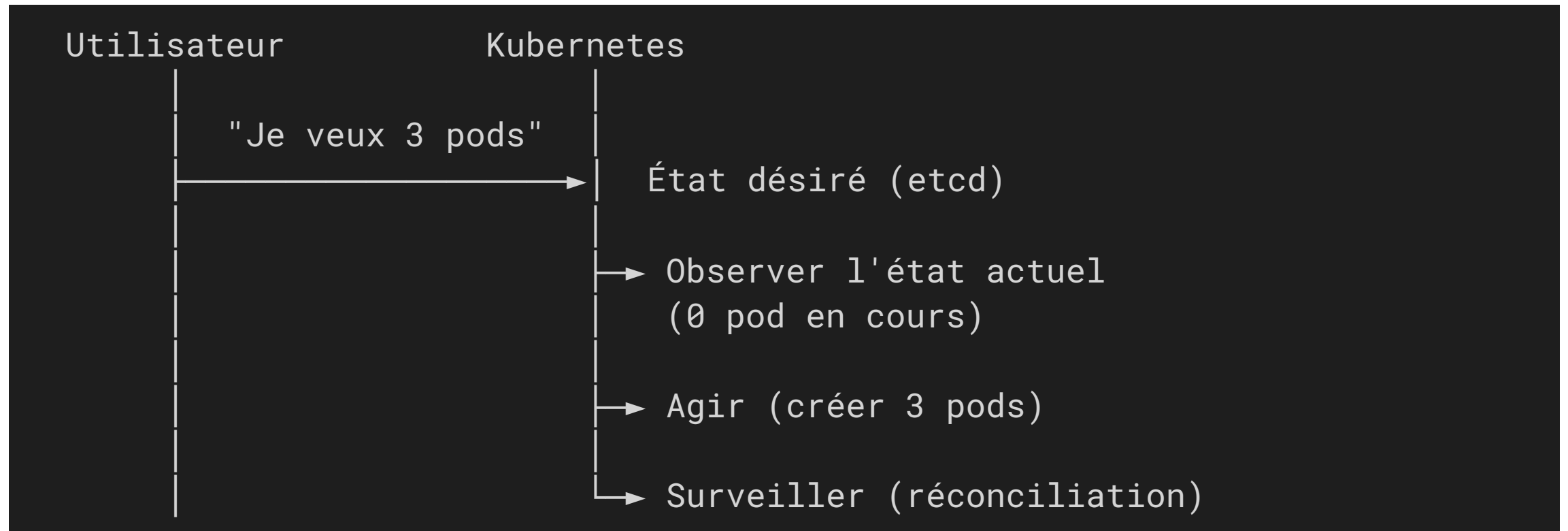
Distributions manag es : EKS (AWS), AKS (Azure), GKE (Google), OpenShift (Red Hat), Rancher, Tanzu.

Architecture de Kubernetes

Comprendre les composants du cluster

Principes de fonctionnement

Modèle déclaratif : l'utilisateur décrit l'état désiré (YAML), Kubernetes converge vers cet état.

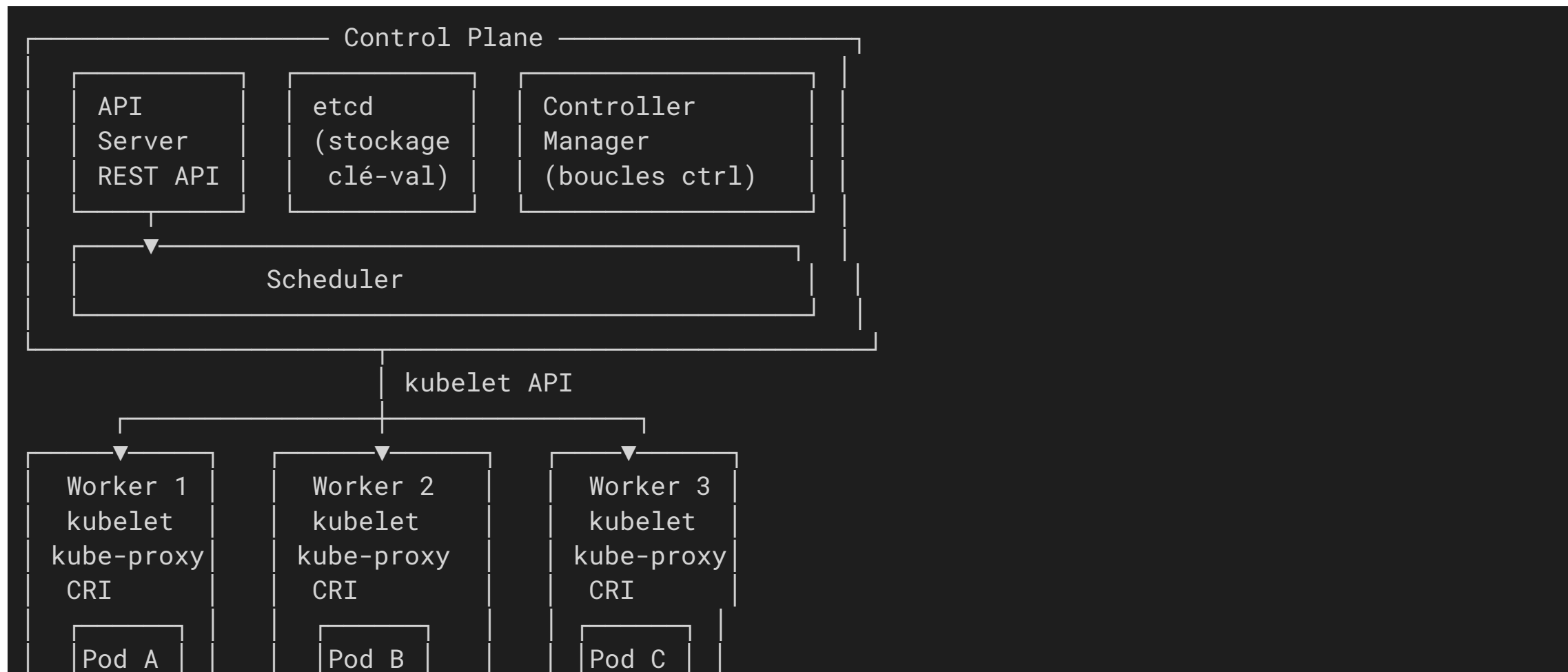


Principes de fonctionnement

Concepts clés :

- **Source de vérité** : etcd (base clé-valeur distribuée)
- **Boucle de réconciliation** : les controllers convergent en permanence
- **API REST** centralisée : tout passe par l'API Server
- **Extensible** : CRDs, Operators, Admission Webhooks

Architecture – Vue d'ensemble



Composants du Control Plane

Composant	Rôle
kube-apiserver	Point d'entrée REST unique. AuthN, AuthZ, Admission Control. Stateless et scalable horizontalement.
etcd	Base clé-valeur distribuée (Raft). Source de vérité du cluster. Sauvegardes critiques.
kube-controller-manager	Boucles de contrôle : ReplicaSet, Deployment, Node, Job, ServiceAccount, EndpointSlice...
kube-scheduler	Assigne les pods non planifiés aux nœuds. Phases Filter + Score.
cloud-controller-manager	(Optionnel) Intégration avec le cloud provider (LoadBalancer, volumes, routes).

Composants du Control Plane

Composants par nœud (workers et control-plane) :

Composant	Rôle
kubelet	Agent qui pilote les conteneurs via le CRI. Communique avec l'API Server.
kube-proxy	Programme iptables/IPVS/nftables pour les Services. Peut être remplacé par eBPF (Cilium).
Container Runtime	containerd, CRI-O, ou autre runtime CRI.

Container Engine (CRI)

CRI = Container Runtime Interface : contrat gRPC entre kubelet et le runtime.

```

kubelet —CRI gRPC—> containerd —OCI—> runc —> conteneur
                    (ou CRI-O)           (ou crun,
                                         kata, gVisor)
  
```

Container Engine (CRI)

Runtime	Description
containerd	Le plus utilisé. Léger, extrait de Docker. Standard sur la plupart des distros.
CRI-O	Conçu pour Kubernetes (Red Hat). Utilisé par OpenShift.
Docker (dockershim)	Retiré depuis Kubernetes 1.24. Utilisez containerd.
Kata Containers	Conteneurs dans des micro-VMs (isolation forte).
gVisor	Sandbox utilisateur (Google). Sécurité renforcée.

```
# Voir le runtime utilisé par chaque nœud
kubectl get nodes -o wide
# La colonne CONTAINER-RUNTIME indique : containerd://1.7.x
```

Container Network Interface (CNI)

CNI = standard de plugin réseau. Kubernetes ne gère **pas** le réseau lui-même.

Modèle réseau Kubernetes :

- Chaque Pod a une **IP unique** dans le cluster
- Tous les Pods peuvent communiquer **sans NAT** entre eux
- Tous les Nodes peuvent joindre tous les Pods
- L'IP que voit un Pod est la même que celle vue par les autres

Container Network Interface (CNI)

Plugin CNI	Caractéristiques
Calico	Routing BGP, NetworkPolicies avancées, eBPF dataplane disponible
Cilium	Basé sur eBPF, observabilité (Hubble), Service Mesh intégré
Flannel	Simple, VXLAN, peu de fonctionnalités avancées
Weave Net	Mesh chiffré, multicast, en perte de vitesse
AWS VPC CNI	IP de pod = IP VPC AWS (EKS)
Azure CNI	IP de pod = IP VNet Azure (AKS)

Container Storage Interface (CSI)

CSI = standard pour les drivers de stockage. Découplé du cœur de Kubernetes.

```
Pod → PVC → PV → StorageClass → CSI Driver → Stockage  
                                           (EBS, Ceph,  
                                           NFS, GCE PD)
```

Container Storage Interface (CSI)

Fonctionnalités CSI :

- **Dynamic Provisioning** : création automatique de volumes
- **Snapshots** : VolumeSnapshot, VolumeSnapshotClass
- **Cloning** : copier un PVC depuis un autre
- **Expansion** : redimensionner un volume à chaud
- **Topology-aware** : placer le pod là où le volume existe

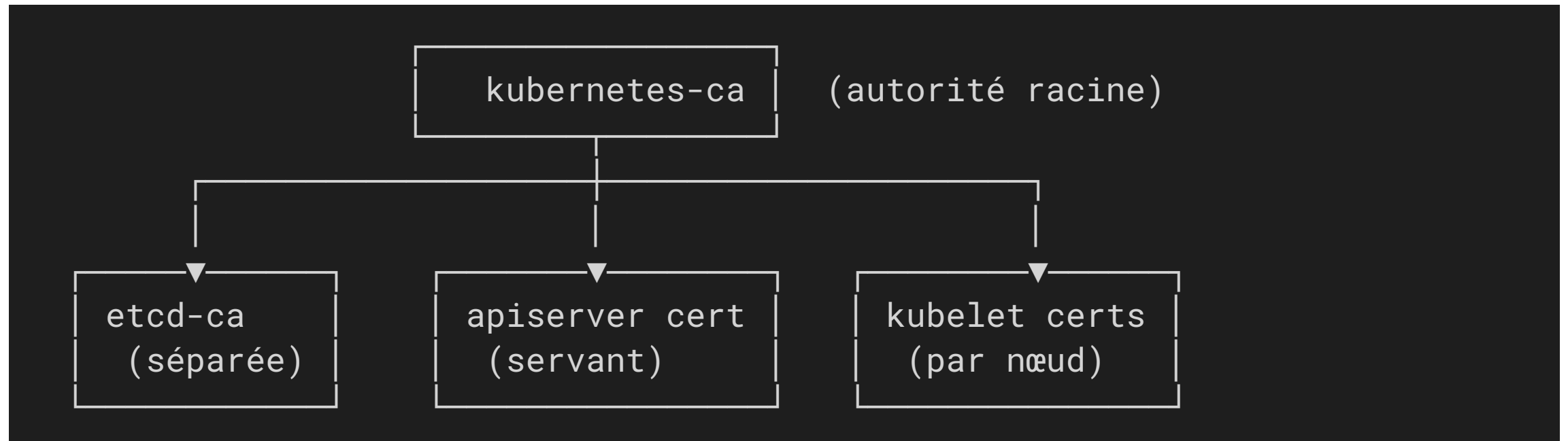
Container Storage Interface (CSI)

Drivers populaires :

Driver	Usage
<code>ebs.csi.aws.com</code>	AWS EBS
<code>disk.csi.azure.com</code>	Azure Disk
<code>pd.csi.storage.gke.io</code>	GCE Persistent Disk
<code>rook-ceph</code>	Ceph distribué
<code>nfs.csi.k8s.io</code>	NFS classique
<code>local.csi.k8s.io</code>	Stockage local (HostPath, LVM)

Certificats et PKI

Kubernetes utilise une **PKI X.509 interne** pour authentifier les composants.



Certificats et PKI

Certificats clés :

Certificat	Usage
<code>apiserver.crt</code>	TLS du kube-apiserver
<code>apiserver-kubelet-client.crt</code>	apiserver → kubelet
<code>apiserver-etcd-client.crt</code>	apiserver → etcd
<code>front-proxy-client.crt</code>	Aggregation Layer
<code>kubelet-client.crt</code>	kubelet → apiserver (par nœud)
<code>sa.key</code> / <code>sa.pub</code>	Signature des tokens de ServiceAccount

Rotation : `kubeadm certs check-expiration` / `kubeadm certs renew all`.

Les certificats expirent par défaut à **1 an**.

Installation de Kubernetes

Du KaaS au cluster bare-metal HA

Kubernetes as a Service (KaaS)

Le cloud provider gère le control plane. Vous gérez les workloads.

Service	Spécificités
Amazon EKS	Control plane managé, intégration IAM, VPC CNI, Fargate (serverless)
Azure AKS	Intégration Azure AD, Azure CNI, virtual nodes (ACI)
Google GKE	Le plus mature, Autopilot (entièrement managé), VPC-native
DigitalOcean DOKS	Simple, prix prévisible
OVHcloud Managed K8s	Européen, RGPD-friendly
Scaleway Kapsule	Européen, intégration LoadBalancer Scaleway

Kubernetes as a Service (KaaS)

Avantages KaaS :

- Pas de gestion d'etcd, ni de patches du control plane
- SLA contractuel (typiquement 99.95 %)
- Intégration native avec les services cloud (IAM, LoadBalancer, stockage)

Limites :

- Coût mensuel (control plane facturé sauf chez GKE jusqu'à 1 cluster gratuit)
- Moins de contrôle (versions imposées, fenêtres de maintenance)

Installation manuelle — kubeadm

kubeadm est l'outil officiel pour bootstrapper un cluster « à la main ».

```
# Sur le futur control-plane
sudo kubeadm init \
  --pod-network-cidr=10.244.0.0/16 \
  --control-plane-endpoint=k8s-lb.corp.local:6443 \
  --upload-certs

# Configurer kubectl pour l'utilisateur courant
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Installer un CNI (ex: Calico)
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.27.0/manifests/calico.yaml

# Joindre un worker (commande affichée par kubeadm init)
sudo kubeadm join k8s-lb.corp.local:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

Outils d'installation

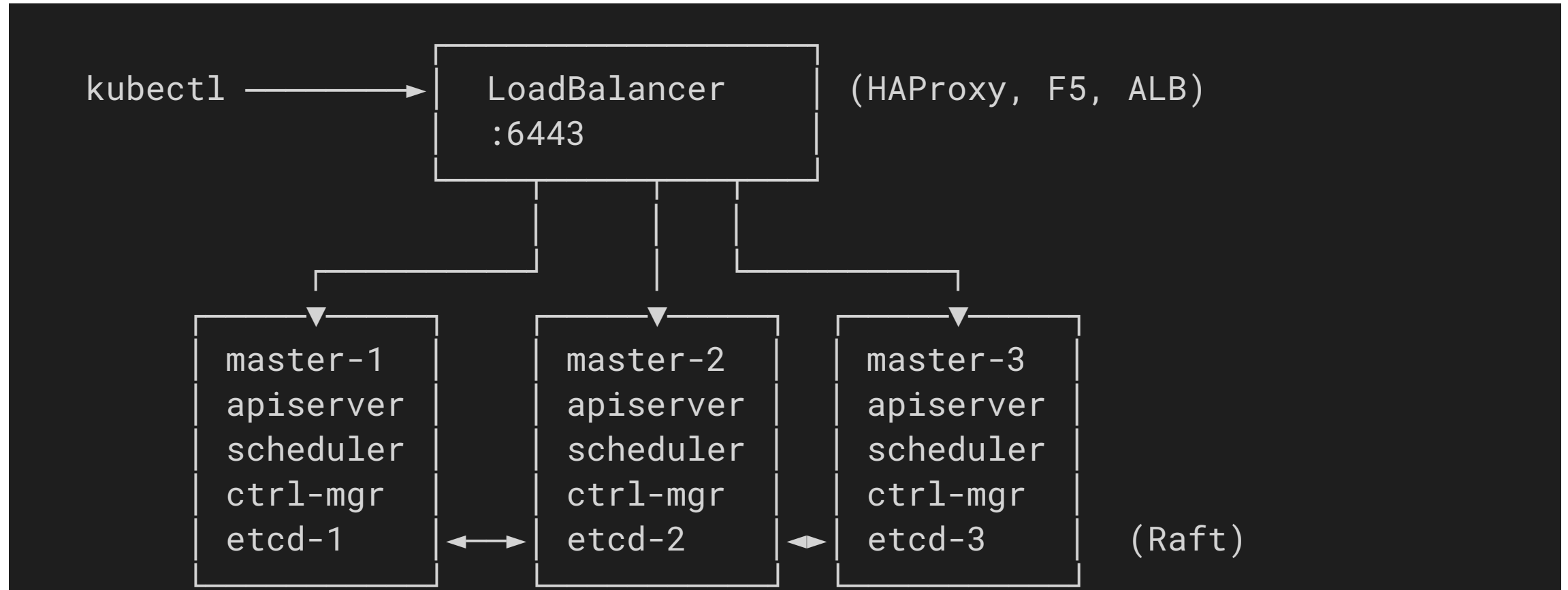
Outil	Cas d'usage
kubeadm	Bootstrap officiel, production on-premise
Kind	« Kubernetes in Docker » – lab, CI, ce qu'on utilise en formation
Minikube	Mono-nœud local, dev rapide
k3s	Distribution légère (Rancher Labs) – edge, IoT, ARM
k0s	Distribution mono-binaire (Mirantis)
MicroK8s	Snap Ubuntu, dev et edge
Rancher / RKE2	Plateforme multi-cluster avec UI
Cluster API	Provisionner des clusters comme du code (CRD)
kOps	Génère des clusters sur AWS/GCP (en perte de vitesse)
Talos Linux	OS immuable dédié à Kubernetes

Kind – pourquoi ?

```
# Installer Kind
brew install kind                # macOS
go install sigs.k8s.io/kind@latest # depuis source
# Créer un cluster multi-nœuds
cat <<EOF | kind create cluster --config=-
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
  - role: worker
EOF
kubectl get nodes
```

Haute Disponibilité (HA)

Cluster de production = **plusieurs control-planes** + **etcd répliqué** + **Load Balancer**.



Utilisation de Kubernetes

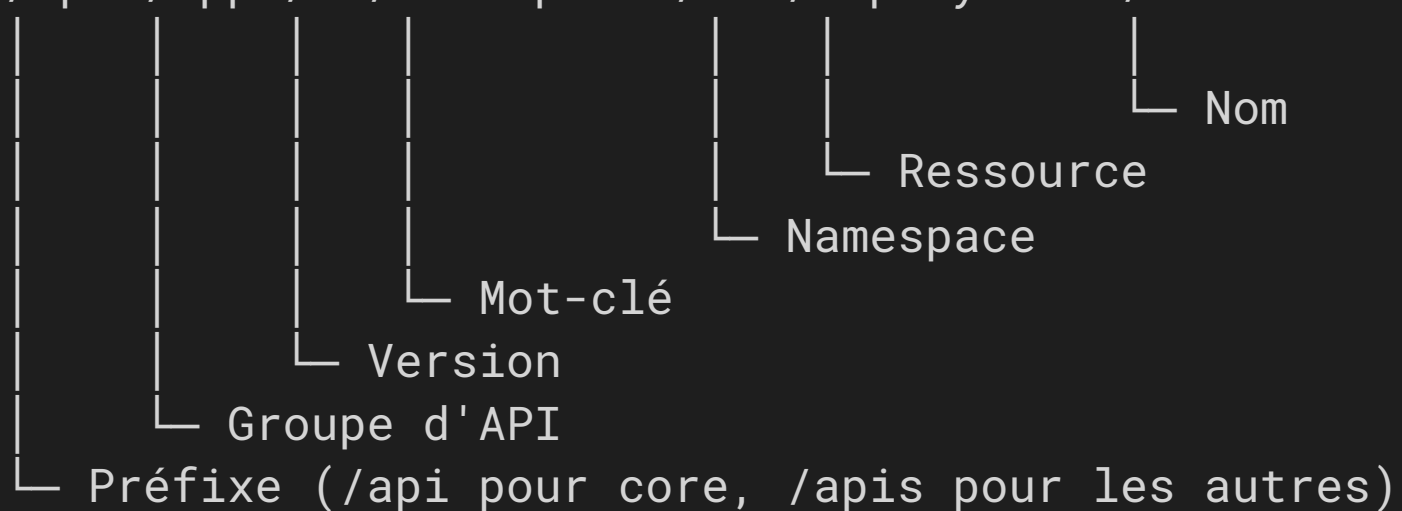
L'API et les ressources

Kubernetes API

Toute interaction avec Kubernetes = appel REST vers l'API Server.

Anatomie d'une URL d'API :

```
https://api.cluster.local:6443/apis/apps/v1/namespaces/dev/deployments/web
```



- Préfixe (/api pour core, /apis pour les autres)
- Groupe d'API
- Version
- Mot-clé
- Namespace
- Ressource
- Nom

Kubernetes API

Verbes HTTP / actions RBAC :

HTTP	Verb K8s	Description
GET	get / list / watch	Lecture (avec long-polling)
POST	create	Création
PUT	update	Remplacement complet
PATCH	patch	Modification partielle (JSON Patch, Strategic Merge)
DELETE	delete / deletecollection	Suppression

Groupes d'API et versions

```
# Lister tous les groupes et versions
kubectl api-versions

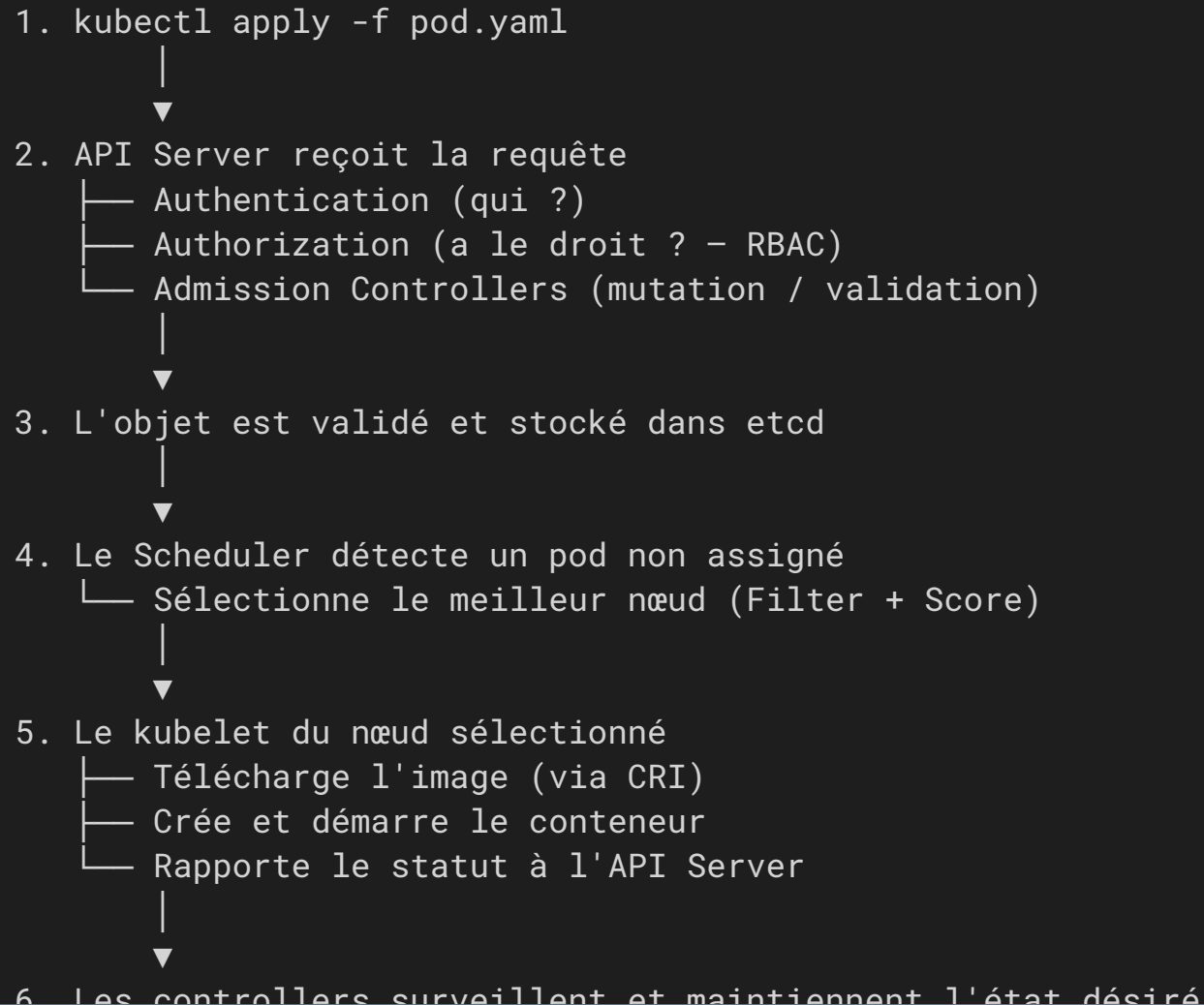
# Lister toutes les ressources avec leur groupe
kubectl api-resources

# Voir le schéma d'une ressource
kubectl explain deployment.spec.template.spec.containers
```


Groupes d'API et versions

Groupe	Ressources
`` (core)	Pod, Service, Node, Namespace, ConfigMap, Secret, PersistentVolume, ServiceAccount
apps/v1	Deployment, ReplicaSet, StatefulSet, DaemonSet
batch/v1	Job, CronJob
networking.k8s.io/v1	Ingress, NetworkPolicy, IngressClass
rbac.authorization.k8s.io/v1	Role, RoleBinding, ClusterRole, ClusterRoleBinding
storage.k8s.io/v1	StorageClass, CSIDriver, VolumeAttachment
policy/v1	PodDisruptionBudget
autoscaling/v2	HorizontalPodAutoscaler
apiextensions.k8s.io/v1	CustomResourceDefinition

Le flux d'une requête `kubectl apply`



Pods – l'unité de base

Un **Pod** est la plus petite unité déployable. Un ou plusieurs conteneurs qui partagent **réseau, volumes, namespace IPC/PID**.

```
apiVersion: v1
kind: Pod
metadata:
  name: mon-pod
  labels:
    app: demo
spec:
  containers:
    - name: app
      image: nginx:1.25
      ports:
        - containerPort: 80
      resources:
        requests: { cpu: "100m", memory: "128Mi" }
        limits: { cpu: "500m", memory: "256Mi" }
      livenessProbe:
        httpGet: { path: /, port: 80 }
```

Deployment — gérer des Pods stateless

Le **Deployment** gère un **ReplicaSet** qui gère les **Pods**. Mises à jour déclaratives + rollback.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  selector:
    matchLabels: { app: web }
  template:
    metadata:
      labels: { app: web }
    spec:
```

Secret — données sensibles

```
# Création
```

```
kubectl create secret generic db-creds \  
  --from-literal=username=admin \  
  --from-literal=password='S3cret!'
```

```
# Secret TLS
```

```
kubectl create secret tls mon-tls --cert=tls.crt --key=tls.key
```

```
# Secret depuis un fichier .env
```

```
kubectl create secret generic app-secret --from-env-file=.env
```

Secret — données sensibles

```
# Injection comme variable d'environnement
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef: { name: db-creds, key: password }

# ... ou comme fichier monté
volumes:
  - name: tls
    secret: { secretName: mon-tls }
```

“ Les Secrets sont **encodés** en base64, pas chiffrés. Activez **Encryption at Rest** côté apiserver et utilisez un secret manager

Ingress — exposer des services HTTP/HTTPS

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx          # IngressClass (NGINX, Traefik, HAProxy...)
  tls:
    - hosts: [app.formation.local]
      secretName: app-tls
  rules:
    - host: app.formation.local
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service: { name: api-svc, port: { number: 8080 } }
          - path: /
            pathType: Prefix
            backend:
```

IngressClass et contrôleurs

L'**Ingress** est juste une ressource — il faut un **Ingress Controller** pour l'implémenter.

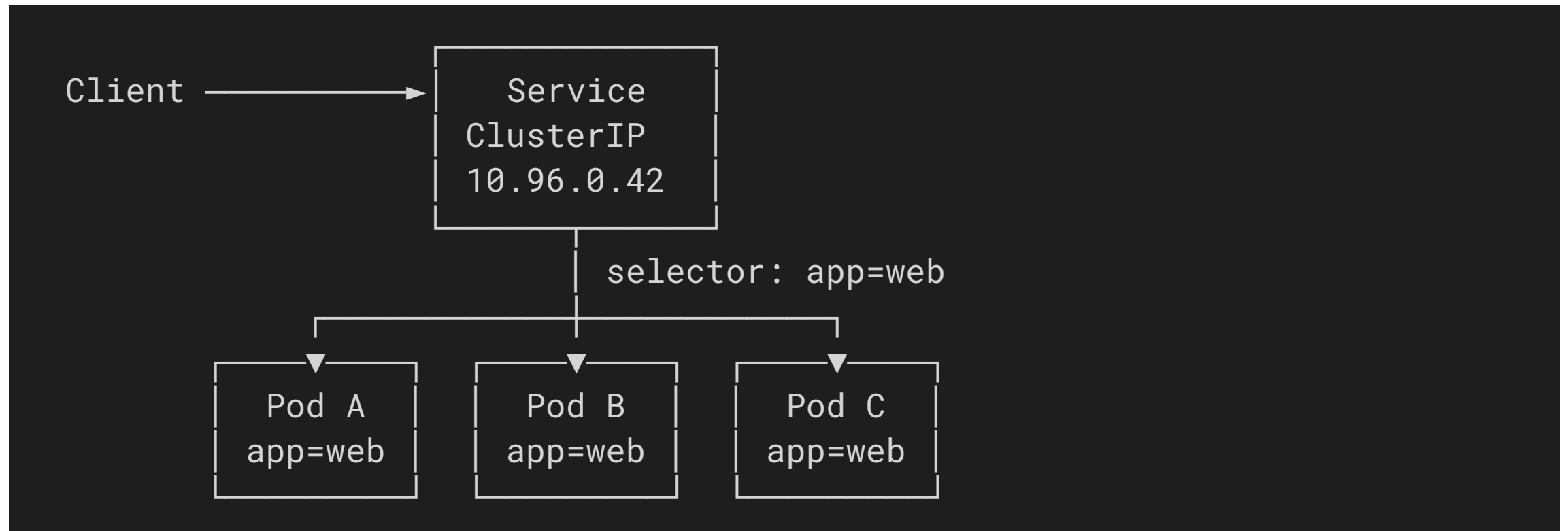
```
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: nginx
  annotations:
    ingresscontroller.k8s.io/is-default-class: "true"
spec:
  controller: k8s.io/ingress-nginx
```


IngressClass et contrôleurs

Contrôleur	Caractéristiques
ingress-nginx	Le plus utilisé, basé sur NGINX
Traefik	Configuration dynamique, support CRDs avancées
HAProxy	Performance, ACL complexes
Contour	Basé sur Envoy, géré par VMware
Istio Gateway	Si vous utilisez Istio (vu jour 2)
Cloud LB Controllers	AWS ALB, GCP GCE, Azure App Gateway

Services — exposer des Pods

Un **Service** est une abstraction réseau stable devant un ensemble de Pods sélectionnés par labels.



Types de Services

```
apiVersion: v1
kind: Service
metadata:
  name: mon-svc
spec:
  type: ClusterIP                # à changer selon le besoin
  selector: { app: mon-app }
  ports:
    - port: 80                  # port du Service
      targetPort: 8080          # port du conteneur
      protocol: TCP
```

Types de Services

Type	Accès	Cas d'usage
ClusterIP	Interne au cluster	Communication inter-services (par défaut)
NodePort	IP de tout nœud + port 30000-32767	Dev, tests, exposition basique
LoadBalancer	IP externe (cloud provider)	Production cloud
ExternalName	Alias DNS vers un service externe	Migration, services hors cluster
Headless (clusterIP: None)	Pas d'IP, retourne directement les Pods	StatefulSet, découverte custom

Endpoints et EndpointSlices

Derrière chaque Service, Kubernetes maintient la liste des Pods cibles.

```
# Ancien modèle (Endpoints) – limité à ~1000 entrées
```

```
kubectl get endpoints mon-svc
```

```
# Nouveau modèle (EndpointSlices) – par défaut depuis 1.21
```

```
kubectl get endpointslices -l kubernetes.io/service-name=mon-svc
```

Endpoints et EndpointSlices

```
# Un EndpointSlice (créé automatiquement)
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  name: mon-svc-abc12
  labels:
    kubernetes.io/service-name: mon-svc
addressType: IPv4
ports:
  - port: 8080
    protocol: TCP
endpoints:
  - addresses: ["10.244.0.5"]
    conditions: { ready: true }
    nodeName: worker-1
  - addresses: ["10.244.1.7"]
```

Service Discovery et DNS

Kubernetes fournit un **DNS interne** (CoreDNS) automatique.

```
<service-name>.<namespace>.svc.cluster.local
```

```
# Depuis n'importe quel pod
curl http://mon-svc.dev.svc.cluster.local
curl http://mon-svc.dev           # raccourci
curl http://mon-svc              # dans le même namespace
```

Service Discovery et DNS

Headless Service — retourne directement les IPs de Pods :

```
spec:  
  clusterIP: None  
  selector: { app: mon-db }
```

Idéal pour les **StatefulSets** : `pod-0.mon-db.dev.svc.cluster.local`,
`pod-1.mon-db.dev.svc.cluster.local`.

StatefulSet — applications avec état

Pour BDD, caches, queues : **identité stable, stockage dédié, ordre de démarrage.**

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres-headless      # Headless service requis
  replicas: 3
  selector:
    matchLabels: { app: postgres }
  template:
    metadata:
      labels: { app: postgres }
    spec:
      containers:
        - name: postgres
          image: postgres:16
          volumeMounts:
```

DaemonSet — un pod par nœud

Idéal pour les agents système : logs, monitoring, CNI, sécurité.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
  namespace: logging
spec:
  selector:
    matchLabels: { app: fluent-bit }
  template:
    metadata:
      labels: { app: fluent-bit }
    spec:
      tolerations:
        # Tolérer les taints du control-plane
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule
      containers:
        - name: fluent-bit
          image: fluent/fluent-bit:3
          volumeMounts:
```

Job et CronJob — tâches batch

Job — exécution unique jusqu'à succès :

```
apiVersion: batch/v1
kind: Job
metadata: { name: migration }
spec:
  completions: 1           # nombre de succès requis
  parallelism: 1           # exécutions en parallèle
  backoffLimit: 3          # tentatives avant échec
  ttlSecondsAfterFinished: 3600
  template:
    spec:
      restartPolicy: OnFailure
      containers:
        - { name: migrate, image: my-migrator:1.0 }
```

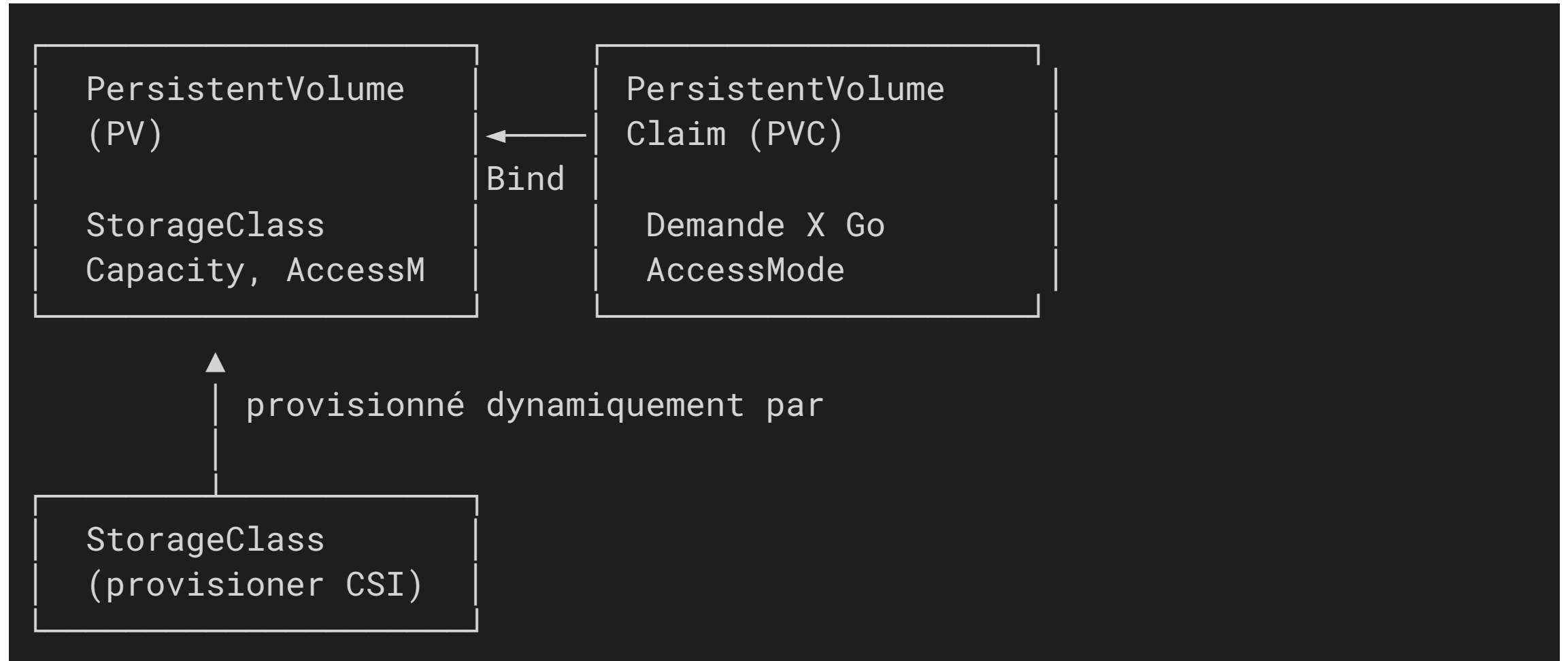
Volumes – types

Type	Persistance	Cas d'usage
emptyDir	Vie du Pod	Cache, fichiers temporaires partagés entre conteneurs
hostPath	Vie du nœud	Accès filesystem nœud (dev only, dangereux en prod)
PersistentVolume	Indépendant	BDD, données applicatives
configMap / secret	-	Fichiers de configuration
projected	-	Combine plusieurs sources
csi	Selon driver	Stockage cloud/distribué via CSI

Volumes – types

```
# emptyDir – partage entre conteneurs
volumes:
- name: shared
  emptyDir: {}
- name: shared-memory
  emptyDir:
    medium: Memory          # tmpfs (RAM)
    sizeLimit: 256Mi
```

PV, PVC et StorageClass



PV, PVC et StorageClass

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: { name: fast-ssd }
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  iops: "3000"
reclaimPolicy: Delete          # ou Retain
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

PV, PVC et StorageClass

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: app-data }
spec:
  accessModes: [ReadWriteOnce]    # RWO, ROX, RWX, RWOP
  storageClassName: fast-ssd
  resources: { requests: { storage: 20Gi } }
```


Access Modes et Reclaim Policy

Access Mode	Code	Description
ReadWriteOnce	RWO	Lecture/écriture par un seul nœud
ReadOnlyMany	ROX	Lecture seule par plusieurs nœuds
ReadWriteMany	RWX	Lecture/écriture par plusieurs nœuds (NFS, CephFS)
ReadWriteOncePod	RWOP	Lecture/écriture par un seul pod (1.27+)

Access Modes et Reclaim Policy

Reclaim Policy	Comportement à la suppression du PVC
Delete	Le PV et le stockage sous-jacent sont supprimés
Retain	Le PV est conservé en Released, à nettoyer manuellement
Recycle	(déprécié) – wipe basique puis re-disponible

Access Modes et Reclaim Policy

```
kubectl get pv  
kubectl get pvc  
kubectl get sc
```

Requests, Limits et QoS

```
resources:
  requests:                # garanti par le Scheduler
    cpu: "100m"            # 0.1 CPU
    memory: "128Mi"
  limits:                  # plafond appliqué par le kernel
    cpu: "500m"
    memory: "256Mi"
```

Requests, Limits et QoS

Classes de QoS (calculées par Kubernetes) :

Classe	Condition	Comportement
Guaranteed	requests = limits pour CPU et mémoire, sur tous les conteneurs	Priorité maximale, dernier tué (OOMKiller)
Burstable	au moins une request définie, ≠ Guaranteed	Peut consommer au-delà des requests si disponible
BestEffort	aucune request ni limit	Premier tué en cas de pression

Requests, Limits et QoS

Pièges classiques :

- **Pas de memory limit** = un Pod peut OOM le nœud entier
- **CPU limit trop bas** = throttling, latence élevée
- **Request = limit en CPU** = excellent pour la prévisibilité

Network Policies

Par défaut, **tous les pods peuvent se parler**. Les NetworkPolicies introduisent un firewall L3/L4.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: deny-all, namespace: prod }
spec:
  podSelector: {}                # tous les pods du namespace
  policyTypes: [Ingress, Egress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-frontend-to-api, namespace: prod }
spec:
  podSelector:
    matchLabels: { app: api }
  ingress:
    - from:

```

Network Policies — exemples avancés

Autoriser tout le DNS sortant (souvent oublié) :

```
egress:
- to:
  - namespaceSelector:
      matchLabels: { kubernetes.io/metadata.name: kube-system }
    podSelector:
      matchLabels: { k8s-app: kube-dns }
ports:
- { port: 53, protocol: UDP }
- { port: 53, protocol: TCP }
```


Network Policies — exemples avancés

Autoriser depuis un autre namespace :

```
ingress:  
  - from:  
    - namespaceSelector:  
      matchLabels: { team: frontend }
```

Network Policies — exemples avancés

Bloquer un CIDR particulier (egress) :

```
egress:  
  - to:  
    - ipBlock:  
      cidr: 0.0.0.0/0  
      except: [169.254.0.0/16, 10.0.0.0/8]
```